

## Module 7: Privacy, Security issues in Blockchain [5L]:

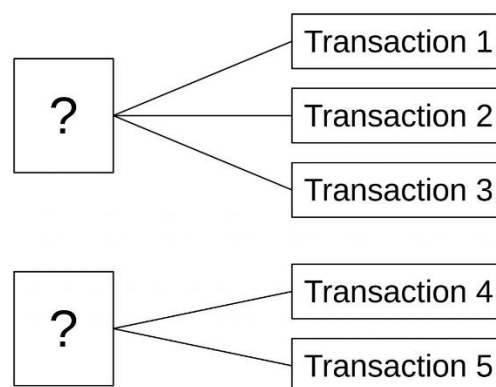
**Pseudo-anonymity vs. anonymity, Zcash and Zk-SNARKS for anonymity preservation, attacks on Blockchains – such as Sybil attacks, selfish mining, 51% attacks - the advent of Algorand, and Sharding based consensus algorithms to prevent these.**

### Pseudo-anonymity vs. anonymity

Anonymity means that the identity of a person is unknown and its actions are not trackable.

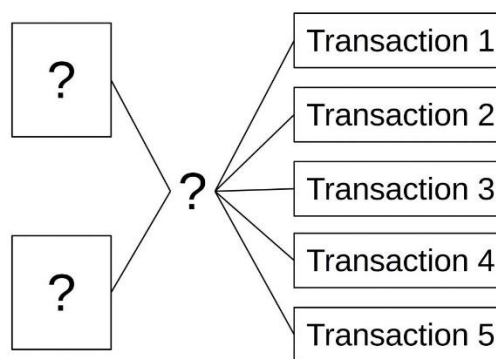
Pseudonymity, however, is a much weaker form. It means that a person's identity is unknown, but it is possible to assign actions to the same person.

The first figure shows a situation where the person's identity is unknown. But all the actions the person made can be linked to this person.



**Pseudonymity**

The second figure shows a situation where a person's identity is unknown and it is impossible to link any transaction to a specific person.



**Anonymity**

Blockchains like Bitcoin or Ethereum are pseudonymous instead of anonymous. The address is a pseudonym for the person holding the corresponding private key. Every action done with this address can be traced back to this person, although its identity might be unknown. And since all transactions are stored in the blockchain, it is possible for everybody to trace them back, even though this transaction lies long in the past.

There are so-called privacy coins like Monero or ZCash that allow for real anonymity through cryptography. Here, transactions cannot be linked to individual addresses.

## Zcash and Zk-SNARKS for anonymity preservation

**Zcash** is a **privacy-centric cryptocurrency** (launched in 2016) designed to provide **shielded transactions**, meaning:

- Sender address can be hidden
- Receiver address can be hidden
- Transaction amount can be hidden

Unlike Bitcoin, where all transactions are publicly visible, Zcash offers **optional privacy**.

**zk-SNARKs** are a type of **zero-knowledge proof** that allows one party to prove something is true **without revealing the underlying information**.

**Key properties:**

- **Zero-knowledge** → No sensitive data is revealed
- **Succinct** → Proofs are very small and fast to verify
- **Non-interactive** → Only one message (proof) is needed
- **Arguments of Knowledge** → Ensures the prover actually knows the data

In Zcash, zk-SNARKs allow transactions to be **validated without exposing details**:

**Normal Blockchain (e.g., Bitcoin):**

- Shows sender, receiver, amount

**Zcash Shielded Transaction:**

- Uses zk-SNARK proof to confirm:
  - Sender has sufficient balance
  - Transaction is valid
  - No double-spending

**But does NOT reveal:**

- Who sent the money
- Who received it
- How much was sent

**Types of Zcash Addresses**

- **Transparent addresses (t-address)** → Similar to Bitcoin (public)
- **Shielded addresses (z-address)** → Fully private using zk-SNARKs

**Advantages:**

- Strong privacy guarantees

- Prevents transaction tracing
- Maintains blockchain integrity
- Enables confidential financial systems

#### **Real-world analogy:**

You prove to a bank that you have enough money **without showing your account balance**.

#### **Challenges & Limitations**

- Computationally intensive (though improving)
- Trusted setup required (initially controversial)
- Not all Zcash transactions are shielded (privacy is optional)
- Regulatory concerns due to anonymity

#### **Attacks on Blockchains – such as Sybil attacks**

##### **Major Types of Blockchain Attacks**

###### **1. Consensus-Level Attacks**

###### ✓ **51% Attack**

When a miner/group controls >50% of network hash power

Can:

- Reverse transactions
- Perform double spending
- Example: Smaller PoW chains have been attacked this way

###### ✓ **Selfish Mining**

- A miner secretly builds a private chain and releases it strategically
- Gains unfair rewards and can destabilize consensus

###### ✓ **Long-Range Attack (PoS)**

- In Proof-of-Stake systems, attackers recreate a fake chain from the past
- Exploits weak checkpointing mechanisms

###### **2. Transaction-Based Attacks**

###### ✓ **Double Spending**

- Spending the same cryptocurrency twice
- Exploits confirmation delays

###### ✓ **Finney Attack**

- A miner pre-mines a block and uses it to reverse a transaction
- Works against fast, zero-confirmation payments

###### ✓ **Race Attack**

- Two conflicting transactions are broadcast simultaneously
- Merchant may accept the wrong one

###### **3. Network-Level Attacks**

###### ✓ **Eclipse Attack**

- Attacker isolates a node by controlling its peers

- Feeds false blockchain data

#### ✓ **Sybil Attack**

- Creating many fake identities to influence the network
- Can assist in consensus manipulation

#### ✓ **Routing / BGP Hijacking**

- Manipulating internet routing to delay/block transactions
- Impacts propagation across nodes

### **4. Smart Contract Attacks (Mainly in Ethereum)**

#### ✓ **Reentrancy Attack**

- Contract calls itself before previous execution finishes
- Famous case: The DAO Hack

#### ✓ **Integer Overflow / Underflow**

- Arithmetic errors allow manipulation of balances

#### ✓ **Front Running**

- Attackers see pending transactions and insert their own first

### **5. Cryptographic Attacks**

#### ✓ **Private Key Theft**

- If keys are compromised, funds are lost

#### ✓ **Weak Hashing / Collision Attacks**

- Exploiting vulnerabilities in hash functions (rare in modern systems)

#### ✓ **Quantum Attacks (Future Threat)**

- Could break current cryptographic signatures

### **6. Wallet & User-Level Attacks**

#### ✓ **Phishing Attacks**

- Fake websites steal private keys

#### ✓ **Malware Attacks**

- Keyloggers or clipboard hijackers

#### ✓ **Social Engineering**

- Tricking users into revealing credentials

### **7. Privacy Attacks**

Even privacy coins like Zcash can face:

- Transaction graph analysis
- Timing analysis
- Metadata leakage

#### **Mitigation Strategies**

- Decentralized mining/staking
- Strong consensus mechanisms
- Multi-signature wallets
- Smart contract auditing
- Network encryption and peer diversity

## Selfish mining

Selfish Mining is a technique used by a group of crypto miners who deliberately hide newly mined blocks from the public blockchain. Instead of broadcasting them to the network, they keep them private and continue mining in secret.

To get a head start and release their hidden blocks only when they're ahead of the public chain. If successful, the network will accept their version of the blockchain as the correct one — and ignore the efforts of honest miners.

- This lets selfish miners:
- earn more block rewards,
- slow down the network,
- undermine the fairness of the system,
- And potentially push the network toward centralization.

### How Selfish Mining Works (Step-by-Step)

- A selfish miner (or pool) finds a block — let's say block #1001.
- They do not broadcast it to the network.
- Honest miners keep working on block #1000, unaware that a new block already exists.
- Meanwhile, the selfish pool finds block #1002 — and keeps that one private, too.
- Once their private chain is longer than the public one, they release it.
- The blockchain protocol chooses the longest valid chain — so their version wins.
- Honest miners lose out on rewards, and the selfish pool collects everything.

If selfish mining became widespread:

- It would discourage honest participation in the network.
- Small miners would be pushed out, accelerating centralization.
- Transactions with low fees might be intentionally delayed or ignored.
- The value and credibility of the cryptocurrency could collapse.
- The core principle of decentralization would be undermined.

## 51% attacks - the advent of Algorand

A **51% attack** happens when a single entity (or colluding group) controls **more than 50% of the network's total consensus power**, allowing them to:

- Reverse transactions (double-spending)
- Prevent new transactions from being confirmed
- Temporarily fork the blockchain

This is especially relevant in systems like **Bitcoin**, which rely on **Proof of Work**.

### Limitations of PoW Systems

- In PoW-based blockchains:

- Power = computational hash rate
- Mining pools can potentially centralize power
- Example concern: large mining pools in Bitcoin

### Sharding based consensus algorithms to prevent these

**Sharding** is often proposed as a way to **scale blockchains and reduce the feasibility of 51% attacks**, but it doesn't magically eliminate the threat—it **reshapes how attacks would need to happen**. Let's connect sharding with consensus design clearly.

#### What is Sharding in Blockchain?

**Sharding** splits the network into smaller groups called *shards*, where each shard:

- Processes its own transactions
- Maintains its own state
- Runs a localized consensus

Instead of one global chain, you have **parallel mini-blockchains**

#### Problem: 51% Attack in Sharded Systems

In a sharded system, an attacker doesn't need to control the entire network. Instead:

- They can target a **single shard**
- If they gain majority control in that shard → they can corrupt it

This is called a **"single-shard takeover attack"**

### Sharding-Based Consensus Mechanisms to Prevent Attacks

#### 1. Random Validator Assignment

- Validators are randomly assigned to shards using cryptographic randomness
- Often uses **Verifiable Random Function (VRF)**

Prevents attackers from choosing which shard to attack

#### 2. Frequent Reshuffling

- Validators are periodically reshuffled across shards

Even if an attacker gains influence in one shard, it's temporary

#### 3. Large Committee Size per Shard

- Each shard has many validators

Reduces probability that attacker controls majority in any shard

#### 4. Byzantine Fault Tolerant Consensus in Each Shard

- Shards use protocols based on **Byzantine Fault Tolerance**
- Requires  $\geq 2/3$  honest nodes

Attack requires controlling a large fraction of a shard—not easy

## 5. Cross-Shard Validation / Beacon Chain

- A central chain (beacon chain) monitors shards
- Example: **Ethereum (post-upgrade design)**

Detects malicious shard behavior and enforces global consistency

## 6. Stake-Based Security (PoS + Sharding)

- Validators stake tokens
- Malicious behavior → stake slashed

Economic deterrence against shard takeover

## 7. Data Availability Sampling

- Nodes randomly verify shard data availability

Prevents attackers from hiding invalid data

## Real-World Example

### Ethereum 2.0

- Uses a **Beacon Chain + shard chains**
- Random validator assignment
- Strong slashing penalties
- Cross-shard coordination